



Text Encryption and Decryption using AES Cryptographic Algorithm

Course: Information Security Lab

Group Members: Muhammad Kabir (245-2024) · Muhammad Faizan (18-2024)

UNIVERSITY INFORMATION SECURITY LAB PROJECT

Introduction

Cryptography is the science of protecting information by converting it into a form that only authorized people can read. This project demonstrates how modern encryption keeps data safe.

Encryption

Converts readable text into unreadable, scrambled text using a key

Decryption

Converts encrypted text back to its original readable form

AES Algorithm

A strong, widely-used standard for secure data protection



Problem Statement



Why Encryption is Needed

- Plain text can be read by anyone who accesses it
- Sensitive data — passwords, messages, records — needs protection
- Basic methods like Caesar Cipher are easily broken
- A stronger, modern algorithm is required for real security

⚠ Caesar Cipher shifts letters by a fixed amount — it can be cracked in seconds with modern tools.

Project Objectives

1

Secure Encryption

Encrypt user-entered text using the AES algorithm

2

Accurate Decryption

Restore encrypted text back to its original readable form

3

User-Friendly GUI

Provide a simple graphical interface built with Tkinter

4

Practical Demo

Demonstrate real-world AES cryptography using Python



What is AES?



Advanced Encryption Standard

AES is a modern encryption algorithm adopted worldwide. It was established by the U.S. government and is now used in banking, Wi-Fi security, and secure messaging apps.

Key Sizes

Supports 128, 192, and 256-bit keys

Real-World Use

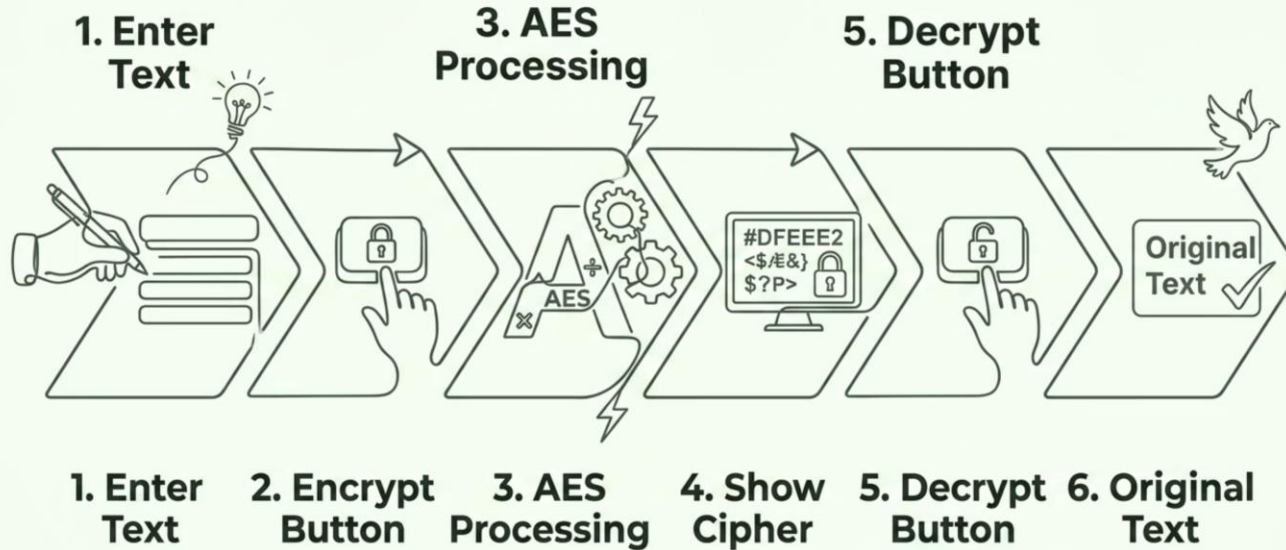
Banking, HTTPS, VPNs, Wi-Fi (WPA2)

Stronger Than Caesar

Cannot be broken by simple frequency analysis

System Workflow

The application follows a clear step-by-step process from user input to encrypted output, and back again through decryption.



This workflow ensures that data is only readable when the correct key is applied — protecting information at every step.



Tools & Technologies



Python

The main programming language used to build the application logic and handle encryption



Tkinter

Python's built-in GUI library for creating buttons, text boxes, and the user interface



PyCryptodome

A Python library that provides AES encryption and decryption functions



AES Algorithm

The core cryptographic standard used to securely encrypt and decrypt user text

Project Features



Text Encryption

Converts plain text into secure cipher text instantly



Text Decryption

Restores cipher text back to its original readable form



GUI Interface

Clean, easy-to-use window with labeled input fields



Clear Button

Resets all fields with a single click for quick reuse



Error Handling

Shows friendly messages when input is invalid or missing



Easy to Use

No technical knowledge needed — just type and click

Application Interface

The GUI provides a clean, intuitive interface for encryption and decryption.



Main GUI Window

Input field, Encrypt and Decrypt buttons, and output display area



Encryption Example

Plain text entered → AES encrypted
output displayed



Decryption Example

Encrypted text entered → original plain
text restored

Source Code Overview



Key Code Components

AES Key

A fixed 16-byte key used by the AES algorithm for encryption and decryption

Encrypt Function

Takes plain text, pads it, and returns encrypted cipher text

Decrypt Function

Takes cipher text and returns the original plain text

GUI Components

Tkinter widgets: Entry, Button, Label, and Text areas

Source Code Overview

```
main.py X
D:\> AES_Project_Kabir_Faizan > src > main.py > decrypt_text
1 from tkinter import *
2 from tkinter import messagebox
3 from Crypto.Cipher import AES
4 from Crypto.Util.Padding import pad, unpad
5 import base64
6
7 KEY = b"1234567890123456"
8
9 def encrypt_text():
10     try:
11         text = input_box.get("1.0", END).strip()
12         if not text:
13             messagebox.showwarning("Warning", "Please enter text.")
14             return
15
16         cipher = AES.new(KEY, AES.MODE_ECB)
17         encrypted = cipher.encrypt(pad(text.encode(), AES.block_size))
18         output_box.delete("1.0", END)
19         output_box.insert(END, base64.b64encode(encrypted).decode())
20     except Exception as e:
21         messagebox.showerror("Error", str(e))
22
23 def decrypt_text():
24     try:
25         text = output_box.get("1.0", END).strip()
26         cipher = AES.new(KEY, AES.MODE_ECB)
27         decrypted = unpad(cipher.decrypt(base64.b64decode(text)), AES.block_size)
28         output_box.delete("1.0", END)
29         output_box.insert(END, decrypted.decode())
30     except:
31         messagebox.showerror("Error", "Invalid encrypted text.")
32
33 def clear_all():
34     input_box.delete("1.0", END)
35     output_box.delete("1.0", END)
36
37 app = Tk()
38 app.title("AES Encryption & Decryption System")
39 app.geometry("800x550")
40
41 Label(app, text="AES Encryption & Decryption System",
42       font=("Arial", 18, "bold")).pack(pady=10)
43
44 Label(app, text="Enter Text").pack()
45 input_box = Text(app, height=10, width=90)
46 input_box.pack(pady=5)
47
48 frame = Frame(app)
49 frame.pack(pady=10)
50
51 Button(frame, text="Encrypt", width=15, command=encrypt_text).grid(row=0, column=0, padx=5)
52 Button(frame, text="Decrypt", width=15, command=decrypt_text).grid(row=0, column=1, padx=5)
53 Button(frame, text="Clear", width=15, command=clear_all).grid(row=0, column=2, padx=5)
54
55 Label(app, text="Output").pack()
56 output_box = Text(app, height=10, width=90)
57 output_box.pack(pady=5)
58
59 app.mainloop()
60
```

Thank You!

We hope this presentation has provided a clear understanding of our AES Encryption and Decryption project, and the importance of robust cryptographic algorithms in securing sensitive information.

For any further questions or to discuss the project in more detail, please feel free to reach out to the team:

MUHAMMAD KABIR (245-2024)

MUHAMMAD FAIZAN (18-2024)

